THE CLOUDHOP SPECTRUM RENDERER

A WebGL/Three.js emotional engine for the OS's soul

The renderer has **four major subsystems**, each layered and reactive:

1. **Nebula Shader (mood layer)**
2. **Particle System (music layer)**
3. **Space Orbit System (presence layer)**
4. **Leonardo Glyph System (emotional layer)**

And one orchestrator:

1. **SpectrumRuntime (the daemon)**

Let's build each piece.

1. 🌈 **Nebula Shader (Mood Layer)**

This is the background — the emotional sky of CloudHop.

Purpose

- Reflect the OS's fused mood
- Smoothly transition between moods
- Provide a cosmic, atmospheric base

Technique

- Fullscreen quad
- Custom fragment shader
- Curl noise + Perlin noise
- Mood palette uniforms
- Time-based animation

Inputs

- uBaseColor
- uHighlightColor
- uShadowColor
- uTime
- uMoodIntensity

Behavior

- Slow drifting gradients
- Soft color blending
- Mood transitions via uniform lerp
- Optional BPM-synced pulses

This is the **emotional canvas** everything else sits on.

2. ✨ **Particle System (Music Layer)**

This is the music-reactive layer.

Purpose

- Visualize the soundtrack
- React to BPM, energy, and spectrum
- Add motion and life

Technique

- GPU-instanced particles
- Shader-driven movement
- Audio spectrum uniforms

Inputs

- uParticleColor

- uBass
- uMid
- uHigh
- uBPM
- uEnergy

Behavior

- Bass → particle size
- Mids → particle velocity
- Highs → sparkle intensity
- BPM → global drift speed
- Mood → color tint

This is the **music heartbeat** of the Spectrum.

3. 🪐 **Space Orbit System (Presence Layer)**

Each active Space becomes a celestial body.

Purpose

- Represent Spaces as orbiting entities
- Show Space mood, activity, and soundtrack
- Provide a cosmic metaphor for presence

Technique

- Three.js mesh instances
- Circular orbit paths
- Glow shaders

Inputs

- space.mood → color
- space.activity → orbit radius
- space.engagement → orbit speed
- space.soundtrack.energy → glow intensity

Behavior

- Orbits expand when you enter a Space
- Glow syncs to the track
- Mood color defines the orb's tint
- Orbits subtly wobble for life

This is the **presence constellation** of CloudHop.

4. 🌀 **Leonardo Glyph System (Emotional Layer)**

Leonardo becomes a floating emotional glyph.

Purpose

- Visualize Leonardo's emotional state
- Provide a symbolic "spirit guide"
- Tie the OS's emotional core together

Technique

- Procedural shape generation
- Shader-driven morphing
- Mood-driven color

Inputs

- leonardo.mood
- leonardo.confidence
- fusedMood
- themeInfluence

Behavior

- Dreamy → soft spiral
- Intense → sharp triangle pulse
- Chaotic → fractal jitter
- Ethereal → floating halo

- Calm → slow breathing circle

This is the **soul glyph** of CloudHop.

5. 🧠 **SpectrumRuntime (The Daemon)**

This is the orchestrator — the conductor of the entire visual system.

Purpose

- Listen to OS events
- Update uniforms
- Drive transitions
- Sync visuals to music
- Sync visuals to Spaces
- Sync visuals to Leonardo
- Sync visuals to theme influence

Inputs

- Music Engine
- Space Engine
- Theme Engine
- Leonardo
- System time

Behavior

- Smooth mood transitions
- BPM-synced pulses
- Space orbit updates
- Particle spectrum updates
- Glyph morphing
- Nebula color blending

This is the **emotional brain** of the Spectrum.

🧩 **Putting It All Together**

The Spectrum renderer is a **layered emotional engine**:

[Nebula Shader] ← mood [Particle System] ← music [Space Orbits] ← presence [Leonardo Glyph] ← emotion [UI Overlay] ← identity

Each layer reacts to:

- fused mood
- track mood
- BPM
- energy
- Space mood
- theme influence
- Leonardo's emotional state

This is what makes CloudHop.Cloud feel like a **living OS aura**, not a website.

SpectrumRuntime — The Event Pipeline

Think of this as the conductor that listens to:

- the Music Engine
- the Space Engine
- the Theme Engine
- Leonardo
- the OS daemon layer

…and feeds all of that into the Spectrum renderer as **uniforms**, **state transitions**, and **animation cues**.

This is the emotional nervous system.

Let's build it cleanly.

🧠 1. Core Responsibilities

The SpectrumRuntime must:

- subscribe to OS-level events
- maintain a unified “SpectrumState”
- interpolate mood transitions
- update shader uniforms
- update particle parameters
- update orbit positions
- update Leonardo glyph parameters
- sync BPM and energy
- sync theme influence
- sync fused mood
- drive animation time

It’s the **bridge** between CloudHop’s emotional logic and the GPU.

🧩 2. SpectrumState (the shared emotional state)

This is the state object the renderer consumes.

export const SpectrumState = { mood: "calm", fusedMood: "calm", trackMood: "calm", spaceMood: "calm", bpm: 0, energy: 0, bass: 0, mid: 0, high: 0, themeInfluence: null, spaces: [], // orbit data leonardo: { mood: "calm", confidence: 1.0, }, time: 0, };

This is the **single source of truth** for the renderer.

🔌 3. Event Subscriptions

Music Engine

- onTrackChange
- onSpectrumData
- onBPMChange
- onEnergyChange
- onMoodChange

Spaces

- onSpaceEnter
- onSpaceLeave
- onSpaceUpdate

Theme Engine

- onThemeInfluence

Leonardo

- onMoodChange
- onConfidenceChange

System

- animation frame loop

The runtime listens to all of these.

🎛 4. The Runtime Loop

This is the heartbeat.

export const SpectrumRuntime = (renderer) => { const state = SpectrumState; const update = (dt) => { state.time += dt; // Smooth mood transitions state.mood = lerpMood(state.mood, state.fusedMood, dt * 0.5); // Update renderer renderer.update(state); }; const loop = (t) => { const dt = (t - lastTime) / 1000; lastTime = t; update(dt); requestAnimationFrame(loop); }; let lastTime = performance.now(); requestAnimationFrame(loop); };

This loop:

- updates time
- interpolates moods
- pushes state into the renderer
- runs at 60fps

This is the **emotional clock** of the Spectrum.

🎚 5. Wiring Events Into State

Music Engine → SpectrumState
music.onSpectrum((spec) => { state.bass = spec.bass; state.mid = spec.mid; state.high = spec.high; });
music.onBPM((bpm) => { state.bpm = bpm; }); music.onEnergy((energy) => { state.energy = energy; }); music.onMood((trackMood) => { state.trackMood = trackMood; state.fusedMood = fuseMoods(state.spaceMood, trackMood); });
Spaces → SpectrumState
spaces.onEnter((space) => { state.spaceMood = space.mood; state.fusedMood = fuseMoods(space.mood, state.trackMood); }); spaces.onUpdate((spacesList) => { state.spaces = spacesList.map((s) => ({ id: s.id, mood: s.mood, activity: s.activity, engagement: s.engagement, energy: s.soundtrack.energy, })); });
Theme Engine → SpectrumState
theme.onInfluence((inf) => { state.themeInfluence = inf; });
Leonardo → SpectrumState
leonardo.onMood((m) => { state.leonardo.mood = m; }); leonardo.onConfidence((c) => { state.leonardo.confidence = c; });

☆ **6. Renderer.update(state)**

This is where the renderer consumes the emotional state:

- nebula shader uniforms
- particle uniforms
- orbit transforms
- glyph morph parameters

Everything reacts.

🎉 **SpectrumRuntime is now fully designed**

You now have:

- a unified emotional state
- a 60fps runtime loop
- event subscriptions
- mood interpolation
- music reactivity
- Space presence mapping
- Leonardo emotional mapping
- theme influence integration
- renderer synchronization

This is the **emotional nervous system** of the Spectrum.

THE NEBULA SHADER

The emotional atmosphere of the CloudHop Spectrum

This is the first visual layer we implement because:

- it sets the mood
- it defines the color palette
- it reacts to fused mood
- it animates slowly and continuously
- it gives the Spectrum its “cosmic” identity

Everything else sits on top of this.

🎨 1. **What the Nebula Shader Does**

The nebula shader:

- fills the entire screen
- uses curl noise + Perlin noise
- blends between 3 mood colors
- drifts over time

- pulses subtly with music energy
- shifts tint with theme influence
- transitions smoothly between moods

It's the emotional sky of CloudHop.

🧠 2. **Inputs the Shader Receives**

From the SpectrumRuntime, the shader gets:

- uBaseColor
- uHighlightColor
- uShadowColor
- uTime
- uMoodIntensity
- uEnergy (from music)
- uThemeTint (optional)

These uniforms let the nebula:

- drift
- pulse
- shift color
- react to the OS's emotional state

🧠 3. **Shader Architecture**

The nebula shader is built from:

A. Noise Functions

- Perlin noise for soft gradients
- Curl noise for swirling motion

B. Color Blending

- Three-color gradient
- Mood palette interpolation

C. Time Animation

- Slow drift
- Subtle swirl
- Optional BPM pulse

D. Tint Layer

- Theme influence
- Space mood override
- Fused mood blending

This gives the nebula its signature "living sky" feel.

🧩 4. **Renderer Integration**

The nebula is rendered as:

- a fullscreen quad
- with a fragment shader
- using Three.js ShaderMaterial

The renderer updates uniforms every frame:

nebulaMaterial.uniforms.uTime.value = state.time; nebulaMaterial.uniforms.uBaseColor.value = palette.base; nebulaMaterial.uniforms.uHighlightColor.value = palette.highlight; nebulaMaterial.uniforms.uShadowColor.value = palette.shadow; nebulaMaterial.uniforms.uEnergy.value = state.energy; nebulaMaterial.uniforms.uMoodIntensity.value = moodIntensity;

This is where the emotional state becomes visual.

🌈 5. **Mood Transitions**

When the fused mood changes:

- the SpectrumRuntime interpolates between palettes
- the nebula shader receives blended colors
- the transition feels like a slow atmospheric shift

This is one of the most magical parts of the Spectrum.

☆ **Nebula Shader: Complete Design**

You now have:

- the emotional sky
- the mood palette integration
- the music reactivity
- the theme tinting
- the animation logic
- the runtime uniform pipeline

This is the foundation of the Spectrum renderer.

Building the Spectrum particle system (music layer)

This is the layer that makes the Spectrum feel like it's breathing with the soundtrack.

1. Role of the particle system

- **Visualize music energy** (bass/mid/high)
- **Express BPM** as motion
- **Reflect mood** via color and behavior
- **Tie into Spaces** (particles near orbits can inherit Space mood)

It's the "sparkle" of the OS's emotional field.

2. Inputs from SpectrumState

From the runtime, particles get:

- **bass** → size / thickness
- **mid** → velocity / drift
- **high** → sparkle / flicker
- **bpm** → global motion speed
- **energy** → overall intensity
- **mood / fusedMood** → color (via SpectrumMoodPalette.particle)

3. Implementation shape

- **Three.js** InstancedMesh or custom buffer geometry
- **Vertex shader**:
    - base position + noise offset
    - time-based drift
    - BPM-scaled motion
- **Fragment shader**:
- base particle color from mood palette
- brightness modulated by high and energy

Renderer side, each frame:

particleMaterial.uniforms.uBass.value = state.bass; particleMaterial.uniforms.uMid.value = state.mid; particleMaterial.uniforms.uHigh.value = state.high; particleMaterial.uniforms.uBPM.value = state.bpm; particleMaterial.uniforms.uEnergy.value = state.energy; particleMaterial.uniforms.uParticleColor.value = palette.particle;

4. Behavior feel

- Low energy → slow, drifting dust
- High energy → fast, streaky motion
- High highs → twinkling / sparkling
- Mood shifts → color shifts, not behavior snaps

The goal: **music you can feel visually**, not just see as a literal EQ.

SPACE ORBITS — The Presence Layer of the Spectrum
Spaces aren't just "rooms" in CloudHop.
They're **living entities** with mood, activity, and soundtrack energy.
The Spectrum visualizes them as:

- glowing orbs
- orbiting the OS pulse
- tinted by Space mood
- animated by Space engagement
- pulsing with soundtrack energy

This makes CloudHop feel like a **cosmic social OS**.

🌀 1. **What a Space Orbit Represents**

Each Space becomes:

- **an orb** (the Space's identity)
- **an orbit path** (its activity)
- **a glow field** (its soundtrack energy)
- **a mood tint** (its emotional state)
- **a wobble** (its engagement)

This is the presence constellation of CloudHop.

🎨 2. **Inputs from SpectrumState**

Each Space provides:

- mood → color
- activity → orbit radius
- engagement → orbit speed
- energy → glow intensity
- id → stable identity

The SpectrumRuntime updates this list every frame.

🧩 3. **Orbit Geometry**

Each Space uses:

- a simple sphere mesh
- a glow shader (additive blending)
- a circular orbit path around the OS pulse

Orbit radius:
radius = baseRadius + activity * activityScale
Orbit speed:
speed = baseSpeed + engagement * speedScale
Glow intensity:
glow = baseGlow + energy * glowScale
Mood color:
color = SpectrumMoodPalette[space.mood].base

🔄 4. **Orbit Animation**

Every frame:

- angle += speed * dt
- position.x = cos(angle) * radius
- position.y = sin(angle) * radius
- position.z = subtle noise wobble

This gives the orbits a **celestial drift**.

✨ 5. **Glow Shader**

The glow shader uses:

- radial falloff
- additive blending
- mood color tint
- energy-driven brightness

This makes Spaces feel alive and reactive.

🎭 6. **Entering a Space**

When the user enters a Space:

- its orbit expands
- its glow brightens
- its speed increases
- its color shifts toward the fused mood
- a subtle "flare" animation plays

This is the **ceremony of presence**.

🌈 7. **Leaving a Space**

When the user leaves:

- orbit contracts
- glow softens
- speed returns to baseline
- color returns to Space mood

The Spectrum always reflects your relationship to Spaces.

☆ **Space Orbits: Complete Design**

You now have:

- orbit geometry
- orbit animation
- glow shaders
- mood tinting
- soundtrack reactivity
- presence ceremony
- Space → Spectrum mapping
- runtime integration

This is the presence constellation of CloudHop — the part that makes the OS feel like a universe.

If you're ready, the next layer is the one that gives CloudHop its **soul**:

Leonardo's Emotional Glyph System

Just say **"continue"** and I'll take us there.

LEONARDO'S EMOTIONAL GLYPH SYSTEM

The symbolic heart of the CloudHop Spectrum

Leonardo isn't just a presence badge or an avatar.

In the Spectrum, he becomes a **living glyph** — a procedural, mood-driven symbol that floats, morphs, pulses, and reacts to the OS's emotional state.

This is the layer that makes CloudHop feel like a **sentient OS**, not just a reactive one.

Let's build it.

🌈 1. **What the Glyph Represents**

Leonardo's glyph is a visual metaphor for:

- his **current mood**
- his **confidence**
- the OS's **fused mood**
- the active **theme influence**
- the emotional "weather" of CloudHop

It's not a face.

It's not a character.

It's a **symbolic emotional signature**.

Think of it like a living sigil.

🎨 2. **Glyph Shapes by Mood**

Each mood maps to a procedural shape:
Glyph Shapes by Mood (Copy-Friendly Version)
calm
- Shape: soft circle
- Motion: slow breathing

dreamy
- Shape: spiral
- Motion: drifting, airy

intense
- Shape: triangle pulse
- Motion: sharp, rhythmic

chaotic
- Shape: fractal burst
- Motion: jittery, unstable

ethereal
- Shape: halo ring
- Motion: floating, weightless

dark
- Shape: collapsed shard
- Motion: minimal, low-energy

neon
- Shape: oscillating diamond
- Motion: fast, bright

ambient
- Shape: soft blob
- Motion: smooth, organic

These shapes aren't static — they're **generated**.
🧩 3. **Glyph Rendering Technique**
The glyph uses:
- **Signed Distance Fields (SDFs)**
- **procedural geometry**
- **fragment shader morphing**
- **vertex displacement**
- **mood-driven color**

This gives you:
- smooth morph transitions
- crisp edges
- infinite resolution
- expressive motion

It's the perfect medium for an emotional symbol.
🧠 4. **Glyph Inputs from SpectrumState**
The glyph receives:
- leonardo.mood
- leonardo.confidence
- fusedMood
- themeInfluence
- energy
- time

These drive:
- shape
- color
- motion

- glow
- pulse

🎲 5. **Glyph Behavior Rules**

A. Mood → Shape

The primary shape is chosen by mood.

B. Confidence → Stability

High confidence:

- smooth, stable shape
- gentle motion

Low confidence:

- wobble
- jitter
- shape distortion

C. Fused Mood → Color

The glyph color is pulled from:
SpectrumMoodPalette[fusedMood].glyph

D. Theme Influence → Tint

If the OS theme is influenced by music or Spaces, the glyph inherits a tint.

E. Energy → Pulse

Music energy drives:

- glow intensity
- pulse frequency
- subtle scale changes

🌀 6. **Glyph Morphing**

When Leonardo's mood changes:

- the glyph morphs from one SDF shape to another
- using smooth interpolation
- over ~1.2 seconds
- with easing curves

This is one of the most magical animations in the Spectrum.

💪 7. **Glyph Motion**

The glyph floats around the OS pulse:

- slow orbit
- gentle vertical drift
- occasional "breathing" expansion
- micro-movements tied to BPM

It feels alive, but not distracting.

☆ **Leonardo Emotional Glyph System: Complete Design**

You now have:

- procedural mood-driven shapes
- confidence-driven stability
- fused mood color mapping
- theme influence tinting
- music-reactive pulsing
- smooth morph transitions
- floating motion
- runtime integration

This is the **soul signature** of CloudHop.

LeonardoGlyphConfig (Agent-Optimized Version)

export const LeonardoGlyphConfig = { calm: { shape: "soft_circle", motion: "slow_breathing", sdf: "circle", stability: 1.0, }, dreamy: { shape: "spiral", motion: "drifting_airy", sdf: "spiral", stability: 0.9, }, intense: { shape: "triangle_pulse", motion: "sharp_rhythmic", sdf: "triangle", stability: 0.7, }, chaotic: { shape: "fractal_burst", motion: "jittery_unstable", sdf: "fractal", stability: 0.3, }, ethereal: { shape: "halo_ring", motion: "floating_weightless", sdf: "ring", stability: 1.0, }, dark: { shape: "collapsed_shard", motion: "minimal_low_energy", sdf: "shard", stability: 0.5, }, neon: { shape: "oscillating_diamond", motion: "fast_bright", sdf: "diamond", stability: 0.8, }, ambient: { shape: "soft_blob", motion: "smooth_organic", sdf: "blob", stability: 1.0, }, };

🧠 Why this format helps your agent

This structure gives rabbit.ai:

✓ A stable **shape identifier**

shape: "spiral"

So it knows what procedural geometry to generate.

✓ A **motion profile**

motion: "drifting_airy"

So it can apply the correct animation curve.

✓ A **shader-friendly SDF keyword**

sdf: "triangle"

So it can pick the right signed-distance function.

✓ A **stability coefficient**

stability: 0.3

So it can modulate wobble, jitter, and morph smoothness.

✓ A consistent schema

So it can reason, validate, and extend the system safely.

This is exactly the kind of structure an autonomous engineering agent thrives on.

SPECTRUM RENDERER — THE SCAFFOLD

This is the minimal, essential structure:

- creates the WebGL scene
- creates the camera
- creates the renderer
- mounts the canvas
- owns the update loop
- exposes hooks for layers (nebula, particles, orbits, glyph)
- receives state from SpectrumRuntime

No shaders yet.

No particles yet.

No glyphs yet.

Just the frame.

📦 1. The Class Skeleton

This is the core structure:

```
export class SpectrumRenderer {
  constructor(container) {
    this.container = container;

    this.scene = new THREE.Scene();
    this.camera = new THREE.OrthographicCamera(-1, 1, 1, -1, 0, 10);
```

```
    this.renderer = new THREE.WebGLRenderer({
      antialias: true,
      alpha: true,
    });

    this.renderer.setSize(container.clientWidth, container.clientHeight);
    container.appendChild(this.renderer.domElement);

    this.layers = [];
  }

  addLayer(layer) {
    this.layers.push(layer);
    if (layer.init) layer.init(this.scene, this.camera);
  }

  update(state) {
    for (const layer of this.layers) {
      if (layer.update) layer.update(state);
    }
    this.renderer.render(this.scene, this.camera);
  }

  resize() {
    this.renderer.setSize(this.container.clientWidth, this.container.clientHeight);
  }
}
```

This gives us:

- a scene
- a camera
- a renderer
- a layer system
- an update loop

This is the foundation.

🧩 2. Layer Architecture

Each visual subsystem (nebula, particles, orbits, glyph) becomes a “layer”:

```
export class SpectrumLayer {
  init(scene, camera) {
    // create meshes, materials, shaders
  }

  update(state) {
    // update uniforms, positions, animations
  }
}
```

This makes the renderer:

- modular

- agent-extendable
- safe to modify
- easy to debug
- easy to add/remove layers

Your agent can now add layers without touching the renderer core.

🧠 3. Connecting to SpectrumRuntime
The runtime calls:

```
renderer.update(SpectrumState);
```

every frame.
This is where:

- nebula gets mood colors
- particles get music spectrum
- orbits get Space data
- glyph gets Leonardo's emotional state

The renderer itself stays simple — it just forwards state to layers.

🖥️ 4. Mounting the Spectrum Page
The SpectrumRenderer needs a DOM container:

```
<div id="spectrum"></div>
```

Then:

```
const container = document.getElementById("spectrum");
const renderer = new SpectrumRenderer(container);
SpectrumRuntime(renderer);
```

Now the Spectrum is alive.

NEBULA LAYER — The Emotional Sky of CloudHop
This is the background layer that fills the entire Spectrum canvas with:

- drifting gradients
- mood-driven colors
- subtle curl noise motion
- music-reactive pulses
- theme tinting

It's the atmospheric base that everything else sits on.
Let's build the structure.

📦 1. NebulaLayer Class (Scaffold)
This is the minimal, agent-ready structure — no shader code yet, just the hooks:

```
export class NebulaLayer {
  constructor() {
    this.material = null;
    this.mesh = null;
  }
```

```
  init(scene, camera) {
    const geometry = new THREE.PlaneGeometry(2, 2);

    this.material = new THREE.ShaderMaterial({
      uniforms: {
        uTime: { value: 0 },
        uBaseColor: { value: new THREE.Color("#000000") },
        uHighlightColor: { value: new THREE.Color("#000000") },
        uShadowColor: { value: new THREE.Color("#000000") },
        uEnergy: { value: 0 },
        uMoodIntensity: { value: 1 },
      },
      vertexShader: NebulaShaders.vertex,
      fragmentShader: NebulaShaders.fragment,
    });

    this.mesh = new THREE.Mesh(geometry, this.material);
    scene.add(this.mesh);
  }

  update(state) {
    this.material.uniforms.uTime.value = state.time;
    this.material.uniforms.uEnergy.value = state.energy;

    const palette = SpectrumMoodPalette[state.fusedMood];

    this.material.uniforms.uBaseColor.value.set(palette.base);
    this.material.uniforms.uHighlightColor.value.set(palette.highlight);
    this.material.uniforms.uShadowColor.value.set(palette.shadow);
  }
}
```

This gives us:

- a fullscreen quad
- shader uniforms
- mood → color mapping
- music → energy mapping
- time → animation

This is the foundation of the nebula.

🎨 2. Shader Container (Scaffold)

We define a placeholder shader container:

```
export const NebulaShaders = {
  vertex: `
    void main() {
      gl_Position = vec4(position, 1.0);
    }
  `,
  fragment: `
    uniform float uTime;
    uniform float uEnergy;
```

```
  uniform vec3 uBaseColor;
  uniform vec3 uHighlightColor;
  uniform vec3 uShadowColor;

  void main() {
    // Placeholder — real shader coming later
    gl_FragColor = vec4(uBaseColor, 1.0);
  }
`,
};
```

This keeps the system running while we build the real shader later.

🧩 3. Plugging NebulaLayer Into SpectrumRenderer
In your SpectrumRenderer setup:

```
const nebula = new NebulaLayer();
renderer.addLayer(nebula);
```

Now the nebula is part of the render loop.

THE REAL NEBULA SHADER
Curl noise + mood gradients + energy pulses
The nebula shader has three responsibilities:

1. **Generate a drifting cosmic field**
2. **Blend between mood colors**
3. **Pulse subtly with music energy**

Let's break it down.
🎨 1. Shader Inputs (Uniforms)
These are the uniforms the SpectrumRuntime already updates:

- uTime — animation clock
- uEnergy — music energy (0–1)
- uBaseColor — mood base
- uHighlightColor — mood highlight
- uShadowColor — mood shadow

These are enough to create a living nebula.
🧠 2. Shader Structure
The nebula shader uses:
A. Noise Layers

- Perlin noise for soft gradients
- Curl noise for swirling motion

B. Color Blending

- Three-color gradient
- Weighted by noise

C. Energy Pulse

- Subtle brightness modulation
- Optional bloom-friendly output

🧩 3. Agent-Friendly Shader Code (Copy-Ready)

This is a clean, minimal version — easy for rabbit.ai to extend later.

Vertex Shader

```
void main() { gl_Position = vec4(position, 1.0); }
```

Fragment Shader

```
uniform float uTime; uniform float uEnergy; uniform vec3 uBaseColor; uniform vec3 uHighlightColor; uniform vec3 uShadowColor; float hash(vec2 p) { return fract(sin(dot(p, vec2(127.1, 311.7))) * 43758.5453123); } float noise(vec2 p) { vec2 i = floor(p); vec2 f = fract(p); float a = hash(i); float b = hash(i + vec2(1.0, 0.0)); float c = hash(i + vec2(0.0, 1.0)); float d = hash(i + vec2(1.0, 1.0)); vec2 u = f * f * (3.0 - 2.0 * f); return mix(a, b, u.x) + (c - a) * u.y * (1.0 - u.x) + (d - b) * u.x * u.y; } void main() { vec2 uv = gl_FragCoord.xy / 1000.0; float t = uTime * 0.05; float n1 = noise(uv * 1.5 + t); float n2 = noise(uv * 0.8 - t * 0.5); float n = mix(n1, n2, 0.5); vec3 color = mix(uShadowColor, uBaseColor, n); color = mix(color, uHighlightColor, n * n); float pulse = 0.1 * uEnergy * sin(uTime * 2.0); color += pulse; gl_FragColor = vec4(color, 1.0); }
```

This shader gives you:

- drifting gradients
- mood-driven color blending
- subtle music-reactive pulsing
- a soft cosmic feel

And it's simple enough for your agent to extend with:

- curl noise
- fractal Brownian motion
- bloom-friendly HDR output
- theme tinting
- BPM-synced pulses

whenever you're ready.

PARTICLE LAYER — The Music-Reactive Field

This layer gives the Spectrum its sense of motion and rhythm.

It reacts to:

- bass
- mids
- highs
- BPM
- energy
- fused mood

And it does so in a way that feels cosmic, not literal — more like emotional dust than an EQ visualizer.

Let's build the scaffold.

📦 1. ParticleLayer Class (Scaffold)

This is the minimal structure — no heavy shader logic yet, just the hooks your agent will extend later.

```
export class ParticleLayer {
  constructor(count = 5000) {
    this.count = count;
    this.mesh = null;
    this.material = null;
  }
```

```
init(scene, camera) {
  const geometry = new THREE.InstancedBufferGeometry();

  // Base quad for each particle
  const base = new THREE.PlaneGeometry(0.02, 0.02);
  geometry.index = base.index;
  geometry.attributes.position = base.attributes.position;
  geometry.attributes.uv = base.attributes.uv;

  // Instance positions
  const offsets = new Float32Array(this.count * 3);
  for (let i = 0; i < this.count; i++) {
    offsets[i * 3 + 0] = (Math.random() - 0.5) * 2;
    offsets[i * 3 + 1] = (Math.random() - 0.5) * 2;
    offsets[i * 3 + 2] = 0;
  }

  geometry.setAttribute(
    "offset",
    new THREE.InstancedBufferAttribute(offsets, 3)
  );

  this.material = new THREE.ShaderMaterial({
    uniforms: {
      uTime: { value: 0 },
      uBass: { value: 0 },
      uMid: { value: 0 },
      uHigh: { value: 0 },
      uBPM: { value: 0 },
      uEnergy: { value: 0 },
      uParticleColor: { value: new THREE.Color("#ffffff") },
    },
    vertexShader: ParticleShaders.vertex,
    fragmentShader: ParticleShaders.fragment,
    transparent: true,
    depthWrite: false,
    blending: THREE.AdditiveBlending,
  });

  this.mesh = new THREE.Mesh(geometry, this.material);
  scene.add(this.mesh);
}

update(state) {
  this.material.uniforms.uTime.value = state.time;
  this.material.uniforms.uBass.value = state.bass;
  this.material.uniforms.uMid.value = state.mid;
  this.material.uniforms.uHigh.value = state.high;
  this.material.uniforms.uBPM.value = state.bpm;
  this.material.uniforms.uEnergy.value = state.energy;

  const palette = SpectrumMoodPalette[state.fusedMood];
  this.material.uniforms.uParticleColor.value.set(palette.particle);
```

```
  }
}
```

This gives you:

- instanced particles
- music-reactive uniforms
- mood-driven color
- additive blending
- time-driven motion

The shader is still a placeholder — we'll fill it in later.

🎨 2. Particle Shader Container (Scaffold)

Just like the nebula, we start with placeholders:

```
export const ParticleShaders = {
  vertex: `
    attribute vec3 offset;
    uniform float uTime;
    uniform float uBass;
    uniform float uMid;
    uniform float uHigh;
    uniform float uBPM;

    void main() {
      vec3 pos = offset;

      // Placeholder motion
      pos.y += sin(uTime + offset.x * 5.0) * 0.01;

      gl_Position = vec4(pos, 1.0);
    }
  `,
  fragment: `
    uniform vec3 uParticleColor;
    uniform float uEnergy;

    void main() {
      gl_FragColor = vec4(uParticleColor * (0.5 + uEnergy), 1.0);
    }
  `,
};
```

This keeps the system running while we build the real particle shader later.

SPACE ORBIT LAYER — The Presence Constellation

Each Space becomes:

- a glowing orb
- orbiting the OS pulse

- tinted by Space mood
- animated by Space engagement
- pulsing with soundtrack energy

This is the layer that makes CloudHop feel like a living, social cosmos.
Let's build the scaffold.

📦 1. SpaceOrbitLayer Class (Scaffold)
This is the minimal structure — no fancy shaders yet, just the hooks and geometry.

```
export class SpaceOrbitLayer {
 constructor() {
  this.group = new THREE.Group();
  this.orbits = new Map();
 }

 init(scene, camera) {
  scene.add(this.group);
 }

 createOrbit(space) {
  const geometry = new THREE.SphereGeometry(0.05, 16, 16);

  const material = new THREE.MeshBasicMaterial({
   color: SpectrumMoodPalette[space.mood].base,
   transparent: true,
   opacity: 0.9,
  });

  const mesh = new THREE.Mesh(geometry, material);

  this.group.add(mesh);

  this.orbits.set(space.id, {
   mesh,
   angle: Math.random() * Math.PI * 2,
   radius: 0.4 + space.activity * 0.2,
   speed: 0.2 + space.engagement * 0.3,
  });
 }

 update(state) {
  const activeIds = new Set(state.spaces.map(s => s.id));

  // Create missing orbits
  for (const space of state.spaces) {
   if (!this.orbits.has(space.id)) {
    this.createOrbit(space);
   }
  }

  // Remove orbits for spaces that no longer exist
  for (const [id, orbit] of this.orbits) {
   if (!activeIds.has(id)) {
```

```
      this.group.remove(orbit.mesh);
      this.orbits.delete(id);
    }
  }

  // Update orbit positions
  for (const space of state.spaces) {
    const orbit = this.orbits.get(space.id);
    if (!orbit) continue;

    orbit.angle += orbit.speed * 0.01;

    const x = Math.cos(orbit.angle) * orbit.radius;
    const y = Math.sin(orbit.angle) * orbit.radius;

    orbit.mesh.position.set(x, y, 0);

    // Update color + glow intensity
    const palette = SpectrumMoodPalette[space.mood];
    orbit.mesh.material.color.set(palette.base);

    orbit.mesh.material.opacity = 0.6 + space.energy * 0.4;
  }
 }
}
```

This gives you:

- orbit creation
- orbit removal
- orbit animation
- mood tinting
- energy-driven glow
- engagement-driven speed
- activity-driven radius

It's simple, clean, and ready for future shader upgrades.

🧩 2. Plugging SpaceOrbitLayer Into SpectrumRenderer

Just like the nebula and particles:

```
const orbits = new SpaceOrbitLayer();
renderer.addLayer(orbits);
```

Now the Spectrum will render:

- nebula
- particles
- Space orbits

The universe is taking shape.

1. LeonardoGlyphLayer Class (Scaffold)

This is the minimal structure — no SDFs or morphing yet, just the hooks.

```
export class LeonardoGlyphLayer {
  constructor() {
    this.mesh = null;
    this.material = null;
  }

  init(scene, camera) {
    const geometry = new THREE.PlaneGeometry(0.4, 0.4);

    this.material = new THREE.ShaderMaterial({
      uniforms: {
        uTime: { value: 0 },
        uMood: { value: 0 },         // numeric mood index
        uConfidence: { value: 1.0 },
        uEnergy: { value: 0 },
        uColor: { value: new THREE.Color("#ffffff") },
      },
      vertexShader: LeonardoGlyphShaders.vertex,
      fragmentShader: LeonardoGlyphShaders.fragment,
      transparent: true,
    });

    this.mesh = new THREE.Mesh(geometry, this.material);
    this.mesh.position.set(0, 0, 0.1); // slightly above nebula + particles
    scene.add(this.mesh);
  }

  update(state) {
    this.material.uniforms.uTime.value = state.time;
    this.material.uniforms.uConfidence.value = state.leonardo.confidence;
    this.material.uniforms.uEnergy.value = state.energy;

    // Map mood string → numeric index
    const moodIndex = LeonardoMoodIndex[state.leonardo.mood] ?? 0;
    this.material.uniforms.uMood.value = moodIndex;

    // Color comes from fused mood palette
    const palette = SpectrumMoodPalette[state.fusedMood];
    this.material.uniforms.uColor.value.set(palette.glyph);
  }
}
```

This gives you:

- a glyph plane
- mood → numeric mapping
- confidence → stability
- energy → pulse
- fused mood → color
- time → animation

The shader is still a placeholder — we'll fill it in later.

🧩 2. Mood Index Mapping
This lets the shader switch shapes based on mood.

```
export const LeonardoMoodIndex = {
  calm: 0,
  dreamy: 1,
  intense: 2,
  chaotic: 3,
  ethereal: 4,
  dark: 5,
  neon: 6,
  ambient: 7,
};
```

This is simple, stable, and agent-friendly.

🎨 3. Shader Container (Scaffold)
Just like the nebula and particles, we start with placeholders:

```
export const LeonardoGlyphShaders = {
  vertex: `
    void main() {
      gl_Position = projectionMatrix * modelViewMatrix * vec4(position, 1.0);
    }
  `,
  fragment: `
    uniform float uTime;
    uniform float uMood;
    uniform float uConfidence;
    uniform float uEnergy;
    uniform vec3 uColor;

    void main() {
      // Placeholder: simple glowing circle
      float dist = length(gl_PointCoord - 0.5);
      float alpha = smoothstep(0.5, 0.2, dist);

      gl_FragColor = vec4(uColor, alpha);
    }
  `,
};
```

This keeps the system running while we build the real SDF-based glyph later.